

A Technical Introduction to Gradient boosting

Section 1

$$F_m(x) = F_{m-1}(x) + v \cdot \gamma_m h_m(x), \quad 0 < v \leq 1, \\ \{ \displaystyle F_m(x) = F_{m-1}(x) + \nu \cdot \gamma_m h_m(x), \quad 0 < \nu \leq 1, \}$$

Section 2

If we considered the continuous case, i.e., where H $\{\displaystyle \{\mathcal{H}\}\}$ is the set of arbitrary differentiable functions on R $\{\displaystyle \mathbb{R}\}$, we would update the model in accordance with the following equations

Section 3

Friedman proposes to modify this algorithm so that it chooses a separate optimal value $\gamma_j m$ $\{\displaystyle \gamma_{jm}\}$ for each of the tree's regions, instead of a single γm $\{\displaystyle \gamma_m\}$ for the whole tree. He calls the modified algorithm "TreeBoost". The coefficients $b_j m$ $\{\displaystyle b_{jm}\}$ from the tree-fitting procedure can be then simply discarded and the model update rule becomes:

Section 4

$$F_m(x) = F_{m-1}(x) - \gamma \sum_{i=1}^n \nabla F_{m-1} L(y_i, F_{m-1}(x_i)) \\ \{ \displaystyle F_m(x) = F_{m-1}(x) - \gamma \sum_{i=1}^n \nabla F_{m-1} L(y_i, F_{m-1}(x_i)) \}$$

Section 5

Informal introduction (This section follows the exposition by Cheng Li.) Like other boosting methods, gradient boosting combines weak "learners" into a single strong learner iteratively. It is easiest to explain in the least-squares regression setting, where the goal is to teach a model F $\{\displaystyle F\}$ to predict values of the form $y^{\wedge} = F(x)$ $\{\displaystyle \hat{y} = F(x)\}$ by minimizing the mean squared error $\frac{1}{n} \sum_{i=1}^n (y^{\wedge}_i - y_i)^2$ $\{\displaystyle \frac{1}{n} \sum_{i=1}^n (\hat{y}_i - y_i)^2\}$, where i $\{\displaystyle i\}$ indexes over some training set of size n $\{\displaystyle n\}$ of actual values of the output variable y $\{\displaystyle y\}$:

Section 6

where γm $\{\displaystyle \gamma_m\}$ is the weight at stage m $\{\displaystyle m\}$. We are usually given a training set $\{(x_1, y_1), \dots, (x_n, y_n)\}$ $\{\displaystyle \{(x_1, y_1), \dots, (x_n, y_n)\}\}$ of known values of x and corresponding values of y . In accordance with the empirical risk minimization principle, the method tries to find an approximation $F^{\wedge}(x)$ $\{\displaystyle \hat{F}(x)\}$ that minimizes the average value of the loss function on the training set, i.e., minimizes the empirical risk. It does so by

starting with a model, consisting of a constant function $F_0(x)$ $\{\displaystyle F_0(x)\}$, and incrementally expands it in a greedy fashion:

Section 7

$$F_m(x) = F_{m-1}(x) + \gamma_m h_m(x), \quad \gamma_m = \arg \min_{\gamma} \sum_{i=1}^n L(y_i, F_{m-1}(x_i) + \gamma h_m(x_i)) \\ \{ \displaystyle F_m(x) = F_{m-1}(x) + \gamma_m h_m(x), \quad \gamma_m = \underset{\{\gamma\}}{\operatorname{arg\,min}} \sum_{i=1}^n L(y_i, F_{m-1}(x_i) + \gamma h_m(x_i)) \}$$

Section 8

Algorithm Many supervised learning problems involve an output variable y and a vector of input variables x , related to each other with some probabilistic distribution. The goal is to find some function $F^{\wedge}(x)$ $\{\displaystyle \hat{F}(x)\}$ that best approximates the output variable from the values of input variables. This is formalized by introducing some loss function $L(y, F(x))$ $\{\displaystyle L(y, F(x))\}$ and minimizing it in expectation:

Section 9

$$F_m(x) = F_{m-1}(x) + \sum_{j=1}^J \gamma_{jm} R_{jm}(x), \quad \gamma_{jm} = \arg \min_{\gamma} \sum_{i \in R_{jm}} L(y_i, F_{m-1}(x_i) + \gamma) \\ \{ \displaystyle F_m(x) = F_{m-1}(x) + \sum_{j=1}^J \gamma_{jm} \mathbf{1}_{R_{jm}}(x), \quad \gamma_{jm} = \underset{\{\gamma\}}{\operatorname{arg\,min}} \sum_{i \in R_{jm}} L(y_i, F_{m-1}(x_i) + \gamma) \}$$

Section 10

Regularization Fitting the training set too closely can lead to degradation of the model's generalization ability, that is, its performance on unseen examples. Several so-called regularization techniques reduce this overfitting effect by constraining the fitting procedure. One natural regularization parameter is the number of gradient boosting iterations M (i.e. the number of base models). Increasing M reduces the error on training set, but increases risk of overfitting. An optimal value of M is often selected by monitoring prediction error on a separate validation data set. Another regularization parameter for tree boosting is tree depth. The higher this value the more likely the model will overfit the training data.

Section 11

$$F_0(x) = \arg \min_{h \in H} \sum_{i=1}^n L(y_i, h(x_i)) \\ \{ \displaystyle F_0(x) = \underset{\{h \in \mathcal{H}\}}{\arg \min} \sum_{i=1}^n L(y_i, h(x_i)) \}$$

Section 12

$$F_m(x) = F_{m-1}(x) - \gamma m \sum_{i=1}^n \nabla F_{m-1} L(y_i, F_{m-1}(x_i)) \\ \{ \displaystyle F_m(x) = F_{m-1}(x) - \gamma_m \sum_{i=1}^n \nabla F_{m-1} L(y_i, F_{m-1}(x_i)) \}$$

$$-_{\{F_{m-1}\}}L(y_{\{i\}},F_{m-1}(x_{\{i\}}))\}$$

Section 13

Stochastic gradient boosting Soon after the introduction of gradient boosting, Friedman proposed a minor modification to the algorithm, motivated by Breiman's bootstrap aggregation ("bagging") method. Specifically, he proposed that at each iteration of the algorithm, a base learner should be fit on a subsample of the training set drawn at random without replacement. Friedman observed a substantial improvement in gradient boosting's accuracy with this modification. Subsample size is some constant fraction f of the size of the training set. When $f = 1$, the algorithm is deterministic and identical to the one described above. Smaller values of f introduce randomness into the algorithm and help prevent overfitting, acting as a kind of regularization. The algorithm also becomes faster, because regression trees have to be fit to smaller datasets at each iteration. Friedman obtained that $0.5 \leq f \leq 0.8$ leads to good results for small and moderate sized training sets. Therefore, f is typically set to 0.5, meaning that one half of the training set is used to build each base learner. Also, like in bagging, subsampling allows one to define an out-of-bag error of the prediction performance improvement by evaluating predictions on those observations which were not used in the building of the next base learner. Out-of-bag estimates help avoid the need for an independent validation dataset, but often underestimate actual performance improvement and the optimal number of iterations.